

FAQ

¿Cual IDE me recomienda utilizar?

Se recomienda la utilización de [Visual Studio Code](#) para el desarrollo de esta tarea.

¿Debo ejecutar docker build cada vez que cambio el código?

No se debe generar la imagen cada vez que se modifica el código. Se recomienda:

1. Crear una carpeta para la tarea #1, por ejemplo: `C:\Users\Nereo\Documents\T1`
2. Generar el contenedor `3-Full-Container`.
3. Mapear la carpeta `C:\Users\Nereo\Documents\T1` al contenedor.

```
cd C:\Users\Nerep\Documents\T1
docker run -p 8888:8888 -i -t -v $(pwd)/.: /t1 bigdatafull /bin/bash
```

Este comando expone la carpeta en el directorio `/t1`. En otras palabras, se trabaja directamente sobre la computadora anfitriona y los cambios se reflejan dinámicamente dentro del contenedor. Luego, para comenzar a trabajar, se ejecuta:

```
source /opt/venv/bin/activate
cd /t1
spark-submit {COMMAND NAME}.py
```

¿Qué archivos podría contener la carpeta?

```
.
├── ./t1
│   ├── ./t1/conftest.py
│   ├── ./t1/data
│   │   ├── ./t1/data/actividad.csv
│   │   ├── ./t1/data/ciclista.csv
│   │   └── ./t1/data/ruta.csv
│   ├── ./t1/doc
│   │   ├── ./t1/doc/README.md
│   │   └── ./t1/doc/images
│   │       ├── ./t1/doc/images/img1.png
│   │       └── ./t1/doc/images/img2.png
│   ├── ./t1/functions
│   │   └── ./t1/functions/functions.py
│   ├── ./t1/main.py
│   └── ./t1/tests.py
```

¿Qué podría contener el archivo conftest.py?

```
import pytest

from pyspark.sql import SparkSession

@pytest.fixture(scope="module")
def spark_session():
    """A fixture to create a Spark Context to reuse across tests."""

    s = SparkSession.builder.appName('pytest-local-spark').master('local') \
        .getOrCreate()

    yield s

    s.stop()
```

¿Cómo se podría organizar el archivo main?

```
# =====
# Imports
# =====

import sys

from pyspark.sql import SparkSession

# =====
# Custom imports
# =====

# from functions import (
#     load_cyclists,
#     load_routes,
#     load_activities,
#     join_data,
#     calculate_totals,
#     get_top_cyclists
# )

# =====
# Constants
# =====

# EXAMPLE_CONSTANT = "VALUE"

# =====
```

```

# Helper functions
# =====

def validate_arguments():
    """
    Validate command-line arguments.
    """

    # TODO:
    # Validate number of arguments
    # Validate files exist
    # Validate file extensions

    pass

# =====
# Main function
# =====

def main():

    # -----
    # Validate arguments
    # -----

    validate_arguments()

    # -----
    # Read command-line arguments
    # -----

    # Example:
    #
    # cyclists_file = sys.argv[1]
    # routes_file = sys.argv[2]
    # activities_file = sys.argv[3]

    # -----
    # Create Spark session
    # -----

    spark = SparkSession.builder \
        .appName("Tarea1") \
        .master("local[*]") \
        .getOrCreate()

    # -----
    # Load input data
    # -----

    # TODO:
    #
    # cyclists_df = ...

```

```

# routes_df = ...
# activities_df = ...

# -----
# Data joins
# -----

# TODO:
#
# joined_df = ...

# -----
# Intermediate aggregations
# -----

# TODO:
#
# totals_df = ...

# -----
# Final calculations
# -----

# TODO:
#
# top_cyclists_df = ...

# -----
# Show results
# -----

# TODO:
#
# top_cyclists_df.show()

# -----
# Export results (optional)
# -----

# TODO:
#
# top_cyclists_df.write...

# -----
# Stop Spark session
# -----

spark.stop()

# =====
# Entry point
# =====

```

```
if __name__ == "__main__":
    main()
```

¿Cómo podría organizar el archivo functions?

```
# =====
# Imports
# =====

from pyspark.sql import DataFrame
from pyspark.sql import SparkSession

from pyspark.sql.functions import (
    col,
    sum,
    avg,
    count,
    desc
)

# =====
# Load functions
# =====

def load_cyclists(
    spark: SparkSession,
    file_path: str
) -> DataFrame:
    """
    Load cyclists CSV file.
    """

    # TODO:
    #
    # Define schema
    # Read CSV
    # Return dataframe

    pass

def load_routes(
    spark: SparkSession,
    file_path: str
) -> DataFrame:
    """
    Load routes CSV file.
    """

    # TODO:
```

```
#
# Define schema
# Read CSV
# Return dataframe

pass

def load_activities(
    spark: SparkSession,
    file_path: str
) -> DataFrame:
    """
    Load activities CSV file.
    """

    # TODO:
    #
    # Define schema
    # Read CSV
    # Return dataframe

    pass

# =====
# Join functions
# =====

def join_data(
    cyclists_df: DataFrame,
    routes_df: DataFrame,
    activities_df: DataFrame
) -> DataFrame:
    """
    Join all input datasets.
    """

    # TODO:
    #
    # Join cyclists with activities
    # Join result with routes
    # Return final dataframe

    pass

# =====
# Aggregation functions
# =====

def calculate_total_kilometers(
    joined_df: DataFrame
) -> DataFrame:
```

```

    """
    Calculate total kilometers by cyclist.
    """

    # TODO:
    #
    # Group data
    # Sum kilometers
    # Return dataframe

    pass

def calculate_daily_average(
    joined_df: DataFrame
) -> DataFrame:
    """
    Calculate average daily kilometers.
    """

    # TODO:
    #
    # Aggregate by date
    # Calculate averages
    # Return dataframe

    pass

def calculate_province_totals(
    joined_df: DataFrame
) -> DataFrame:
    """
    Calculate total kilometers by province.
    """

    # TODO:
    #
    # Group by province
    # Sum kilometers
    # Return dataframe

    pass

# =====
# Ranking functions
# =====

def get_top_cyclists_by_total_km(
    totals_df: DataFrame,
    top_n: int = 5
) -> DataFrame:
    """

```

```

    Return top cyclists by total kilometers.
    """

    # TODO:
    #
    # Order dataframe
    # Limit top N
    # Return dataframe

    pass

def get_top_cyclists_by_daily_average(
    averages_df: DataFrame,
    top_n: int = 5
) -> DataFrame:
    """
    Return top cyclists by daily average.
    """

    # TODO:
    #
    # Order dataframe
    # Limit top N
    # Return dataframe

    pass

# =====
# Utility functions
# =====

def validate_dataframe(
    df: DataFrame
) -> bool:
    """
    Validate dataframe content.
    """

    # TODO:
    #
    # Validate nulls
    # Validate schema
    # Validate duplicates

    pass

```

¿Cómo podría organizar el archivo tests.py?

```
# =====
# Imports
# =====

import pytest

from pyspark.sql import Row

from functions import (
    load_cyclists,
    load_routes,
    load_activities,
    join_data,
    calculate_total_kilometers,
    calculate_daily_average,
    calculate_province_totals,
    get_top_cyclists_by_total_km,
    get_top_cyclists_by_daily_average
)

# =====
# Load function tests
# =====

def test_load_cyclists(spark_session):
    """
    Test cyclist loading function.
    """

    # TODO:
    #
    # Load CSV
    # Validate dataframe is not empty
    # Validate schema
    # Validate row count

    pass

def test_load_routes(spark_session):
    """
    Test routes loading function.
    """

    # TODO

    pass

def test_load_activities(spark_session):
    """
    Test activities loading function.
```

```

"""

# TODO

pass

# =====
# Join tests
# =====

def test_join_data(spark_session):
    """
    Test dataframe joins.
    """

    # -----
    # Create test data
    # -----

    cyclists_data = [
        (1, "John Doe", "San Jose")
    ]

    routes_data = [
        (100, "Route A", 25.5)
    ]

    activities_data = [
        (100, 1, "2026-05-01")
    ]

    # -----
    # Create dataframes
    # -----

    # TODO:
    #
    # cyclists_df = ...
    # routes_df = ...
    # activities_df = ...

    # -----
    # Execute join
    # -----

    # TODO:
    #
    # result_df = ...

    # -----
    # Assertions
    # -----

```

```

# TODO:
#
# assert result_df.count() == 1

pass

# =====
# Aggregation tests
# =====

def test_calculate_total_kilometers(spark_session):
    """
    Test total kilometers aggregation.
    """

    # TODO:
    #
    # Create dataframe
    # Execute aggregation
    # Validate totals

    pass

def test_calculate_daily_average(spark_session):
    """
    Test daily average calculation.
    """

    # TODO

    pass

def test_calculate_province_totals(spark_session):
    """
    Test province totals aggregation.
    """

    # TODO

    pass

# =====
# Ranking tests
# =====

def test_get_top_cyclists_by_total_km(spark_session):
    """
    Test top cyclists ranking by total kilometers.
    """

```

```
# TODO:
#
# Create dataframe
# Execute ranking
# Validate order
# Validate top N

pass

def test_get_top_cyclists_by_daily_average(spark_session):
    """
    Test top cyclists ranking by daily average.
    """

    # TODO

    pass

# =====
# Edge case tests
# =====

def test_empty_dataframe(spark_session):
    """
    Test empty dataframe behavior.
    """

    # TODO

    pass

def test_null_values(spark_session):
    """
    Test null handling.
    """

    # TODO

    pass

def test_duplicate_activities(spark_session):
    """
    Test duplicated activity handling.
    """

    # TODO

    pass
```